Helix OTA

Helix OTA — New Submodule Specifications

> **Document ID:** `HELOTA-SUBMOD-001` **Version:** 1.0.0 **Status:** Active **Last Updated:** 2026-03-05 **Constitution Reference:** HelixConstitution v1 §1–§4, §7.1, §11.4.108 **Organization Pattern:** vasic-digital/HelixDevelopment (dual-hosted: GitHub + GitLab)

Table of Contents

---

1. Introduction & Conventions

1.1 Purpose

This document defines the complete implementation specifications for every new submodule that must be created to deliver the Helix OTA system. Each submodule is a standalone, independently versioned, publicly accessible repository hosted under **both** the `HelixDevelopment` GitHub organization and the `HelixDevelopment` GitLab organization. Repositories are mirrored; the GitHub repository is the canonical source, and GitLab provides CI/CD runners and disaster-recovery redundancy.

1.2 Naming Conventions

Property	Convention	Example
Repository name	`helix-<domain>` (kebab-case)	`helix-ota-server`
Go module path	`dev.helix.ota.<domain>` (dot-separated)	`dev.helix.ota.server`
Package names	lowercase, no underscores	`rolloutengine`, `artifactvalidator`
Docker image	`helix-ota/<domain>`	`helix-ota/server`
Helm chart	`helix-ota-<domain>`	`helix-ota-server`

1.3 Dual-Hosting Model

Every submodule repository exists in two places:

1. **GitHub:** `https://github.com/HelixDevelopment/<repo-name>` — canonical source, pull requests, code review, releases
2. **GitLab:** `https://gitlab.com/HelixDevelopment/<repo-name>` — mirror, CI/CD pipelines, container registry, disaster recovery

The GitLab mirror is updated via a push mirror configured in the GitHub repository settings. All merge requests are processed on GitHub; GitLab pipelines are triggered automatically on push.

1.4 Version Strategy

All Helix OTA submodules follow the **main helix_ota versioning scheme** as defined in the VERSION_ROADMAP. The current target is `1.0.0-mvp`. Submodule versions are tagged as `v1.0.0-mvp.<patch>` for initial releases. Once the MVP stabilizes, submodules transition to independent semantic versioning aligned with the parent project version:

- `1.0.0` → MVP release
- `1.0.1` → Rollback support
- `1.0.2` → Delta updates
- `1.1.0` → Linux support
- `1.2.0` → Windows support
- `2.0.0` → Universal multi-OS

1.5 Test Coverage Requirements

Per HelixConstitution §1.1, every submodule must meet:

Metric	Minimum	Enforcement
Line coverage	85%	CI gate: `go test -coverprofile` + `go tool cover`
Mutation score	75%	CI gate: `go-mutesting` with threshold
Integration test coverage	All public API endpoints	Manual verification in PR
Anti-bluff validation	Every test must demonstrate ACTION, DELTA, POSITIVE evidence, and unique TOKEN	Code review checklist

1.6 Required Documentation per HelixConstitution

Every submodule repository **must** contain the following files at the repository root:

File	Purpose	Audience
`README.md`	Project overview, quickstart, architecture diagram, API surface, contribution guide	Humans (developers, operators)
`CLAUDE.md`	Project-specific instructions for AI coding assistants: coding	AI agents (Claude, Copilot)

File	Purpose	Audience
	conventions, test patterns, common gotchas, module structure, naming rules	
`AGENTS.md`	Multi-agent coordination instructions: which agents work on which packages, merge conflict avoidance, dependency update protocols	AI agents & automation
`CONTRIBUTING.md`	PR workflow, branch naming, commit message format, CI requirements	All contributors
`CHANGELOG.md`	Version history following Keep a Changelog format	All stakeholders

---

2. helix-ota-server

2.1 Overview

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-ota-server`
Repository (GitLab)	`HelixDevelopment/helix-ota-server`
Go module	`dev.helix.ota.server`
Language	Go 1.22+
License	Apache 2.0
Visibility	Public

The Helix OTA Server is the central management service for the entire OTA system. It exposes a REST API for device update checks, artifact management, rollout orchestration, telemetry ingestion, and dashboard operations. The server is a monolithic Go application structured as a set of loosely-coupled service modules, each owning a bounded context.

2.2 Package Structure

```
helix-ota-server/
├── cmd/
│   └── server/
│       └── main.go                       # Application
entry point, DI wiring
├── internal/
│   ├── config/
│   │   ├── config.go                     # Configuration
loading & validation
│   │   └── config_test.go
│   ├── handler/
│   │   ├── update_handler.go             # HTTP handlers
for update endpoints
│   │   ├── device_handler.go             # HTTP handlers
for device endpoints
│   │   ├── rollout_handler.go            # HTTP handlers
for rollout endpoints
│   │   ├── artifact_handler.go           # HTTP handlers
for artifact endpoints
│   │   ├── telemetry_handler.go          # HTTP handlers
for telemetry endpoints
│   │   ├── auth_handler.go               # HTTP handlers
for auth endpoints
│   │   ├── notification_handler.go       # WebSocket/SSE
handler for dashboard push
│   │   └── handler_test.go
│   ├── service/
│   │   ├── update_service.go             # Update business
logic
│   │   ├── device_service.go             # Device
management business logic
│   │   ├── rollout_service.go            # Rollout
orchestration
│   │   ├── artifact_service.go           # Artifact
management
│   │   ├── telemetry_service.go          # Telemetry
processing
│   │   ├── auth_service.go               # Authentication
logic
│   │   ├── notification_service.go       # WebSocket hub
management
│   │   └── service_test.go
│   ├── repository/
│   │   ├── postgres/                     # PostgreSQL
repository implementations
│   │   ├── redis/                        # Redis cache
implementations
│   │   └── repo_test.go
│   ├── model/
│   │   ├── device.go                     # Device domain
```

```
model
│   │   ├── artifact.go                   # Artifact domain
model
│   │   ├── rollout.go                    # Rollout domain
model
│   │   ├── telemetry.go                  # Telemetry event
domain model
│   │   ├── user.go                       # User domain
model
│   │   └── errors.go                     # Domain error
types
│   ├── middleware/
│   │   ├── chain.go                      # Middleware chain
builder
│   │   ├── auth.go                       # JWT + mTLS
authentication
│   │   ├── rbac.go                       # Role-based
access control
│   │   ├── logging.go                    # Request/response
logging
│   │   ├── request_id.go                 # Request ID
propagation
│   │   ├── cors.go                       # CORS
configuration
│   │   └── middleware_test.go
│   └── validation/
│       ├── pipeline.go                   # Validation
pipeline orchestrator
│       ├── structure.go                  # ZIP structure
validation
│       ├── hash.go                       # SHA-256 hash
verification (streaming)
│       ├── signature.go                  # RSA signature
verification
│       ├── compatibility.go              # Device
compatibility check
│       ├── worker_pool.go                # Concurrent
validation workers
│       └── validation_test.go
├── pkg/
│   ├── api/
│   │   ├── requests.go                   # Public API
request types
│   │   ├── responses.go                  # Public API
response types
│   │   └── errors.go                     # Public API error
types
│   └── auth/
│       ├── jwt.go                        # JWT token
management
│       ├── mtls.go                       # mTLS certificate
handling
```

```
│       └── device_identity.go            # Device ID
generation
├── migrations/                            # PostgreSQL
schema migrations
├── go.mod
├── go.sum
├── Makefile
├── Dockerfile
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md
```

2.3 Core Types and Interfaces

```
// go.mod
module dev.helix.ota.server

go 1.22

require (
    // Helix OTA submodules
    dev.helix.ota.artifactvalidator v1.0.0
    dev.helix.ota.rolloutengine     v1.0.0
    dev.helix.ota.deviceidentity    v1.0.0
    dev.helix.ota.updateengine      v1.0.0

    // vasic-digital infrastructure submodules
    digital.vasic/auth              v1.2.0
    digital.vasic/database          v1.3.1
    digital.vasic/cache             v1.1.0
    digital.vasic/observability     v1.0.4
    digital.vasic/security          v1.1.2
    digital.vasic/middleware        v1.2.1
    digital.vasic/config            v1.0.3
    digital.vasic/eventbus          v1.1.0
    digital.vasic/storage           v1.2.0
    digital.vasic/ratelimiter       v1.0.2
    digital.vasic/concurrency       v1.0.1
    digital.vasic/recovery          v1.0.0
    digital.vasic/backgroundtasks   v1.1.0

    // External dependencies
    github.com/go-chi/chi/v5        v5.0.12
    github.com/jackc/pgx/v5         v5.5.3
    github.com/golang-migrate/migrate/v4 v4.17.0
    github.com/go-playground/validator/v10 v10.17.0
    go.opentelemetry.io/otel        v1.22.0
    github.com/google/uuid          v1.6.0
```

```
    go.uber.org/zap                    v1.26.0
)

package service

// UpdateService manages the update lifecycle from the
        server's perspective.
type UpdateService interface {
    CheckForUpdate(ctx context.Context, req
        *api.CheckUpdateRequest)
        (*api.UpdateAvailableResponse, error)
    GetUpdate(ctx context.Context, updateID string)
        (*model.Update, error)
    ListUpdates(ctx context.Context, filter
        *api.UpdateFilter, page *api.Pagination)
        (*api.PaginatedResult[model.Update], error)
}

// DeviceService handles device lifecycle management.
type DeviceService interface {
    Register(ctx context.Context, req
        *api.RegisterDeviceRequest) (*model.Device, error)
    Get(ctx context.Context, deviceID string)
        (*model.Device, error)
    List(ctx context.Context, filter *api.DeviceFilter,
        page *api.Pagination)
        (*api.PaginatedResult[model.Device], error)
    Update(ctx context.Context, deviceID string, req
        *api.UpdateDeviceRequest) (*model.Device, error)
    Decommission(ctx context.Context, deviceID string) error
}

// RolloutService manages the lifecycle of update rollouts.
type RolloutService interface {
    Create(ctx context.Context, req
        *api.CreateRolloutRequest) (*model.Rollout, error)
    Get(ctx context.Context, rolloutID string)
        (*model.Rollout, error)
    List(ctx context.Context, filter *api.RolloutFilter,
        page *api.Pagination)
        (*api.PaginatedResult[model.Rollout], error)
    Update(ctx context.Context, rolloutID string, req
        *api.UpdateRolloutRequest) (*model.Rollout, error)
    Pause(ctx context.Context, rolloutID string) error
    Resume(ctx context.Context, rolloutID string) error
    Halt(ctx context.Context, rolloutID string, reason
        string) error
    GetProgress(ctx context.Context, rolloutID string)
        (*api.RolloutProgress, error)
}

// ArtifactService manages OTA artifact lifecycle.
type ArtifactService interface {
```

```go
    Upload(ctx context.Context, req
        *api.UploadArtifactRequest) (*model.Artifact,
        error)
    Validate(ctx context.Context, artifactID string)
        (*api.ValidationResult, error)
    Get(ctx context.Context, artifactID string)
        (*model.Artifact, error)
    List(ctx context.Context, filter *api.ArtifactFilter,
        page *api.Pagination)
        (*api.PaginatedResult[model.Artifact], error)
    Delete(ctx context.Context, artifactID string) error
    Download(ctx context.Context, artifactID string)
        (*api.DownloadURL, error)
}

// TelemetryService handles device telemetry ingestion and
        analysis.
type TelemetryService interface {
    ReportEvent(ctx context.Context, event
        *api.TelemetryEventRequest) error
    GetOverview(ctx context.Context, filter
        *api.TelemetryFilter) (*api.TelemetryOverview,
        error)
    GetDeviceTelemetry(ctx context.Context, deviceID
        string, filter *api.TelemetryFilter, page
        *api.Pagination)
        (*api.PaginatedResult[model.TelemetryEvent], error)
}

// AuthService handles authentication and token management.
type AuthService interface {
    Login(ctx context.Context, req *api.LoginRequest)
        (*api.TokenPair, error)
    RefreshToken(ctx context.Context, refreshToken string)
        (*api.TokenPair, error)
    ValidateToken(ctx context.Context, accessToken string)
        (*api.TokenClaims, error)
    RegisterDevice(ctx context.Context, certPEM []byte)
        (*api.DeviceCredentials, error)
}
```

2.4 Public API Surface

The server exposes the following REST API endpoints (full specification in `REST_API_SPECIFICATION.md`):

Method	Path	Auth	Description
POST	`/api/v1/auth/login`	None	User login with TOTP 2FA
POST	`/api/v1/auth/refresh`	Refresh token	Token rotation

Method	Path	Auth	Description
POST	`/api/v1/auth/logout`	Bearer JWT	Session invalidation
POST	`/api/v1/devices/register`	mTLS	Device self-registration
GET	`/api/v1/devices`	Bearer JWT	List devices
GET	`/api/v1/devices/{id}`	Bearer JWT	Get device details
DELETE	`/api/v1/devices/{id}`	Bearer JWT (admin)	Decommission device
POST	`/api/v1/updates/check`	mTLS	Device update check
POST	`/api/v1/artifacts/upload`	Bearer JWT (admin/operator)	Upload OTA artifact
GET	`/api/v1/artifacts/{id}/download`	mTLS or JWT	Download artifact
POST	`/api/v1/rollouts`	Bearer JWT (admin/operator)	Create rollout
PATCH	`/api/v1/rollouts/{id}`	Bearer JWT (admin/operator)	Update rollout
POST	`/api/v1/devices/{id}/status`	mTLS	Device status report
WS	`/api/v1/ws/dashboard`	Bearer JWT	Dashboard real-time push

2.5 Dependencies

Internal Helix OTA Submodules:

Submodule	Version	Purpose
`dev.helix.ota.artifactvalidator`	v1.0.0	Server-side artifact validation pipeline
`dev.helix.ota.rolloutengine`	v1.0.0	Rollout decision engine and device selection
`dev.helix.ota.deviceidentity`	v1.0.0	Device certificate verification and ID generation
`dev.helix.ota.updateengine`	v1.0.0	Update lifecycle state machine (server-side)

vasic-digital Infrastructure Submodules:

Submodule	Version	Purpose
`digital.vasic/auth`	v1.2.0	JWT issuance, validation, rotation
`digital.vasic/database`	v1.3.1	PostgreSQL connection pool, migrations
`digital.vasic/cache`	v1.1.0	Redis caching layer (L1+L2)
`digital.vasic/observability`	v1.0.4	Prometheus metrics, structured logging
`digital.vasic/security`	v1.1.2	TLS configuration, certificate management
`digital.vasic/middleware`	v1.2.1	CORS, request ID, recovery middleware
`digital.vasic/config`	v1.0.3	Environment-based configuration
	v1.1.0	

Submodule	Version	Purpose
`digital.vasic/eventbus`		Async event processing
`digital.vasic/storage`	v1.2.0	S3/MinIO artifact storage
`digital.vasic/ratelimiter`	v1.0.2	Per-device and per-IP rate limiting
`digital.vasic/concurrency`	v1.0.1	Goroutine pool for validation workers
`digital.vasic/recovery`	v1.0.0	Panic recovery, graceful shutdown
`digital.vasic/backgroundtasks`	v1.1.0	Rollout evaluation scheduler

External Dependencies: chi/v5 (HTTP router), pgx/v5 (PostgreSQL driver), golang-migrate (schema migration), validator/v10 (input validation), otel (tracing), uuid (ID generation), zap (logging).

2.6 Container Integration

The server is deployed as a Docker container defined in `vasic-digital/containers`. The Dockerfile is multi-stage:

```dockerfile
# Build stage
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /helix-ota-
        server ./cmd/server/

# Runtime stage
FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=builder /helix-ota-server /helix-ota-server
EXPOSE 8080 8443
USER nonroot:nonroot
ENTRYPOINT ["/helix-ota-server"]
```

The `docker-compose.yml` in the `containers` submodule orchestrates the server alongside PostgreSQL, Redis, and MinIO. The server container receives configuration via environment variables and mounts the TLS certificates as secrets.

2.7 Documentation Requirements

File	Required Sections
`README.md`	Architecture diagram, API surface table, quickstart with docker-compose, configuration reference, vasic-digital dependency table
`CLAUDE.md`	Go coding conventions (DI via constructors, no global state, error wrapping with `%w`), test patterns (unique evidence TOKEN per test), mock generation commands
`AGENTS.md`	Package ownership map (which agent edits which package), merge conflict zones (handler/service boundary), dependency update protocol

2.8 Test Coverage Requirements

Category	Target	Tool
Unit tests	85% line coverage, 700+ test cases	Go `testing`, testify
Mutation tests	75% mutation score	`go-mutesting`
Integration tests	All API endpoints against testcontainers	`testcontainers-go`
Load tests	10K concurrent devices, p95 < 500ms	k6
Security tests	Auth bypass, signature forgery, replay	OWASP ZAP + custom

3. helix-ota-client-sdk

3.1 Overview

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-ota-client-sdk`
Repository (GitLab)	`HelixDevelopment/helix-ota-client-sdk`
Go module	`dev.helix.ota.clientsdk`
Language	Go 1.22+ (cross-compiled; gomobile for Android AAR)
License	Apache 2.0
Visibility	Public

The Helix OTA Client SDK is a platform-agnostic Go library providing device-side update logic: update checking, artifact download with resume, SHA-256 + RSA verification, installation orchestration, and status reporting. It is consumed by the Android client (via gomobile AAR bindings) and can be embedded directly in any Go-based client for future OS targets (Linux daemon, Windows service).

3.2 Package Structure

```
helix-ota-client-sdk/
├── cmd/
│   └── gomobile/
│       └── main.go                    # gomobile bind
entry point
├── pkg/
│   ├── checker/
│   │   ├── checker.go                 # UpdateChecker
implementation
│   │   └── checker_test.go
│   ├── downloader/
│   │   ├── manager.go                 # DownloadManager
with resume support
│   │   ├── chunked.go                 # HTTP Range-based
chunked download
│   │   ├── throttle.go                # Bandwidth
throttling
│   │   └── downloader_test.go
│   ├── verifier/
│   │   ├── hash.go                    # SHA-256 hash
verification
│   │   ├── signature.go               # RSA-4096
```

```
signature verification
│   │   ├── zipstructure.go               # ZIP structure
validation
│   │   └── verifier_test.go
│   ├── reporter/
│   │   ├── reporter.go                   # StatusReporter
with offline queue
│   │   ├── retry.go                      # Exponential
backoff retry
│   │   └── reporter_test.go
│   ├── statemachine/
│   │   ├── machine.go                    # Update lifecycle
state machine
│   │   ├── states.go                     # State and event
definitions
│   │   ├── transitions.go                # Valid transition
map
│   │   └── statemachine_test.go
│   ├── config/
│   │   ├── config.go                     # Client
configuration
│   │   └── config_test.go
│   ├── mtls/
│   │   ├── certmanager.go                # mTLS certificate
loading and rotation
│   │   ├── pinning.go                    # Server
certificate pinning
│   │   └── mtls_test.go
│   └── clientsdk/
│       ├── sdk.go                        # Main SDK entry
point (facade)
│       └── sdk_test.go
├── bindings/
│   └── android/
│       ├── HelixOtaSdk.java             # Generated Java
interface
│       └── build.gradle                  # AAR build
configuration
├── go.mod
├── go.sum
├── Makefile                              # Includes `make
aar` target
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md
```

3.3 Core Types and Interfaces

```go
// go.mod
module dev.helix.ota.clientsdk
```

```go
go 1.22

require (
    // Helix OTA submodules
    dev.helix.ota.updateengine       v1.0.0
    dev.helix.ota.artifactvalidator  v1.0.0
    dev.helix.ota.deviceidentity     v1.0.0

    // External
    golang.org/x/crypto              v0.18.0
    github.com/google/uuid           v1.6.0
)

package clientsdk

// Config holds the client SDK configuration.
type Config struct {
    ServerURL        string        // e.g., "https://
        api.helix-ota.io"
    DeviceID         string        // Unique device
        identifier
    CheckInterval    time.Duration // Default: 4 * time.Hour
    DownloadDir      string        // Directory for
        temporary download files
    MaxDownloadBytes int64         // Maximum artifact size
        (default: 2 GB)
    ThrottleBytesPerSec int64      // Bandwidth throttle (0
        = unlimited)
    PublicKeyPEM     string        // Embedded RSA public
        key for signature verification
    CACertPEM        string        // CA certificate for
        server pinning
    DeviceCertPEM    string        // Device mTLS client
        certificate
    DeviceKeyPEM     string        // Device mTLS private
        key
    OfflineQueueSize int           // Max queued status
        reports (default: 1000)
}

// UpdateInfo represents an available update from the
        server.
type UpdateInfo struct {
    ArtifactID   string `json:"artifact_id"`
    Version      string `json:"version"`
    DownloadURL  string `json:"download_url"`
    SHA256       string `json:"sha256"`
    SizeBytes    int64  `json:"size_bytes"`
    SignatureURL string `json:"signature_url"`
    Mandatory    bool   `json:"mandatory"`
    Deadline     *time.Time `json:"deadline,omitempty"`
}
```

```go
// UpdateChecker polls the OTA server for available updates.
type UpdateChecker interface {
    Check(ctx context.Context) (*UpdateInfo, error)
}

// DownloadManager handles resumable artifact downloads.
type DownloadManager interface {
    Download(ctx context.Context, url string,
        expectedSHA256 string, expectedSize int64,
        progressChan chan<- int) (*DownloadResult, error)
    Cancel() error
}

// DownloadResult contains the path and verification result
        of a completed download.
type DownloadResult struct {
    FilePath string
    SHA256   string
    Size     int64
}

// StatusReporter sends device status updates to the server.
type StatusReporter interface {
    Report(ctx context.Context, status UpdateStatus) error
    Flush(ctx context.Context) error  // Flush offline queue
}

// UpdateStatus represents a device's current update status.
type UpdateStatus struct {
    DeviceID        string     `json:"device_id"`
    ArtifactID      string     `json:"artifact_id"`
    State           string
        `json:"state"` // DOWNLOADING, VERIFYING, APPLYING,
        REBOOTING, SUCCESS, FAILED
    ProgressPercent int        `json:"progress_percent"`
    ErrorCode       string     `json:"error_code,omitempty"`
    ErrorMessage    string
        `json:"error_message,omitempty"`
    Timestamp       time.Time  `json:"timestamp"`
}

// HelixOtaSDK is the main facade for the client SDK.
// It is safe for concurrent use and manages the update
        lifecycle.
type HelixOtaSDK interface {
    // CheckForUpdate queries the server for available
        updates.
    CheckForUpdate(ctx context.Context) (*UpdateInfo, error)

    // DownloadUpdate downloads the artifact with resume
        support.
    DownloadUpdate(ctx context.Context, info *UpdateInfo,
        progressChan chan<- int) (*DownloadResult, error)
```

```go
    // VerifyArtifact performs SHA-256 and RSA signature verification.
    VerifyArtifact(ctx context.Context, filePath string, info *UpdateInfo) error

    // ReportStatus sends a status update to the server.
    ReportStatus(ctx context.Context, status UpdateStatus) error

    // CurrentState returns the current update lifecycle state.
    CurrentState() State
}
```

3.4 gomobile Android AAR Generation

The SDK produces an Android AAR via `gomobile bind`:

```makefile
# Makefile excerpt
.PHONY: aar
aar: generate
    gomobile bind -target=android -o bindings/android/helix-ota-sdk.aar \
        -androidapi=28 \
        dev.helix.ota.clientsdk/pkg/clientsdk
```

The AAR exposes the `HelixOtaSDK` interface as Java classes. The Android app imports the AAR and wraps it with Kotlin coroutines for asynchronous operation.

3.5 mTLS Integration

```go
package mtls

// CertManager manages device mTLS certificates for server communication.
type CertManager struct {
    deviceCert *x509.Certificate
    deviceKey  crypto.PrivateKey
    caCert     *x509.Certificate
    serverPins []string  // SHA-256 hashes of trusted CA public keys
}

// NewCertManager loads certificates from the configured paths.
func NewCertManager(cfg Config) (*CertManager, error) { /* ... */ }

// HTTPClient returns an *http.Client configured for mTLS with server pinning.
func (cm *CertManager) HTTPClient() (*http.Client, error) {
```

```go
    tlsConfig := &tls.Config{
        MinVersion: tls.VersionTLS13,
        MaxVersion: tls.VersionTLS13,
        Certificates:
         []tls.Certificate{cm.deviceCertPair()},
        VerifyConnection: cm.verifyServerPin,
    }
    return &http.Client{Transport:
         &http.Transport{TLSClientConfig: tlsConfig}}, nil
}
```

3.6 Dependencies

Submodule	Purpose
`dev.helix.ota.updateengine`	State machine definitions and update lifecycle types
`dev.helix.ota.artifactvalidator`	Client-side artifact verification functions
`dev.helix.ota.deviceidentity`	Device identity generation for registration

3.7 Container Integration

The Client SDK itself is not containerized — it is a library. However, it is used within the `helix-ota-android` system app and in future Linux/Windows client containers. The SDK's configuration is designed to be environment-variable driven, enabling container-friendly deployment of Go-based clients that embed it.

3.8 Documentation Requirements

File	Required Sections
`README.md`	Quickstart (Go import), gomobile AAR build instructions, configuration reference, state machine diagram, error handling guide
`CLAUDE.md`	gomobile constraints (no `chan` in exported types, no `interface{}` in AAR-visible signatures), test patterns for network mocking
`AGENTS.md`	Package ownership, AAR generation workflow, version bump protocol

3.9 Test Coverage Requirements

Category	Target	Notes
Unit	85% line, 60+ tests	State machine, download manager, verifier, reporter
Mutation	75% score	Critical: hash comparison, signature verification, state transitions
Integration	httptest server	Download resume, retry backoff, mTLS handshake
gomobile	AAR compile check	CI must verify AAR builds successfully

---

4. helix-ota-android

4.1 Overview

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-ota-android`
Repository (GitLab)	`HelixDevelopment/helix-ota-android`
Go module	N/A (Kotlin/Android project)
Language	Kotlin 1.9+, Android SDK 35
Min SDK	28 (Android 9)
Target SDK	35 (Android 15)
License	Apache 2.0
Visibility	Public

The Helix OTA Android Client is a system-privileged application pre-installed in `/system/priv-app/HelixOtaClient/`. It integrates with Android's `update_engine` daemon via Binder IPC to apply A/B partition updates. The app wraps the `helix-ota-client-sdk` gomobile AAR for server communication and adds Android-specific orchestration: WorkManager scheduling, foreground services, notification management, and `update_engine` integration.

4.2 Package Structure

```
helix-ota-android/
├── app/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/com/helix/ota/client/
│   │   │   │   ├── HelixOtaApplication.kt        #
Application, DI setup
│   │   │   │   ├── di/
│   │   │   │   │   ├── AppModule.kt               #
Singleton bindings
│   │   │   │   │   ├── NetworkModule.kt            #
OkHttp/Retrofit
│   │   │   │   │   └── SdkModule.kt                # Client
SDK AAR wrapper
│   │   │   │   ├── engine/
│   │   │   │   │   ├── UpdateEngineProxy.kt        # Binder
IPC wrapper
│   │   │   │   │   ├── UpdateEngineCallback.kt     #
IUpdateEngineCallback impl
│   │   │   │   │   └── BootControlHelper.kt        # Boot
control HAL
│   │   │   │   ├── service/
│   │   │   │   │   ├── UpdateCheckWorker.kt        #
WorkManager periodic worker
│   │   │   │   │   ├── DownloadService.kt          #
Foreground download service
│   │   │   │   │   ├── InstallService.kt           #
update_engine orchestration
│   │   │   │   │   └── ReportingService.kt         # Status
reporting
│   │   │   │   ├── notification/
│   │   │   │   │   └── NotificationHelper.kt       #
Notification channels
│   │   │   │   ├── state/
│   │   │   │   │   ├── OtaStateMachine.kt          #
Android-side state machine
│   │   │   │   │   └── OtaState.kt                 # State/
event definitions
│   │   │   │   ├── receiver/
│   │   │   │   │   ├── BootCompletedReceiver.kt    #
Schedule checks after reboot
│   │   │   │   │   └── NetworkChangeReceiver.kt    # Resume
downloads
│   │   │   │   └── ui/
│   │   │   │       ├── MainActivity.kt             #
```

```
Settings & manual check
│   │   │   │       ├── UpdateAvailableActivity.kt   #
Update confirmation
│   │   │   │       └── InstallProgressActivity.kt   #
Install progress
│   │   │   ├── res/
│   │   │   └── AndroidManifest.xml
│   │   ├── test/                                   # Unit
tests (JVM)
│   │   └── androidTest/                            #
Instrumentation tests
│   ├── build.gradle.kts
│   └── libs/
│       └── helix-ota-sdk.aar                       #
gomobile-generated AAR
├── build.gradle.kts
├── settings.gradle.kts
├── gradle.properties
├── Makefile                                        # AAR
copy + APK build
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md
```

4.3 Core Types

```kotlin
// OtaState.kt — Android-side update state machine
enum class OtaState {
    IDLE,
    CHECKING,
    UPDATE_AVAILABLE,
    DOWNLOADING,
    DOWNLOAD_PAUSED,
    VERIFYING,
    INSTALLING,
    INSTALL_VERIFYING,
    INSTALL_FINALIZING,
    REBOOTING,
    COMMITTING,
    SUCCEEDED,
    FAILED
}

// UpdateCheckWorker.kt — WorkManager periodic worker
class UpdateCheckWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker(context, params) {
    override suspend fun doWork(): Result {
        val sdk = HelixOtaSdk.newInstance(/* config */)
```

```kotlin
        val updateInfo = sdk.checkForUpdate()
        // ...
    }

    companion object {
        fun schedule(context: Context, intervalHours: Long
        = 4) {
            val request =
        PeriodicWorkRequestBuilder<UpdateCheckWorker>(intervalHours,
        TimeUnit.HOURS)
                .setConstraints(Constraints.Builder()
                    .setRequiredNetworkType(NetworkType.CONNECTED)
                    .setRequiresBatteryNotLow(true)
                    .build())
                .setBackoffCriteria(BackoffPolicy.EXPONENTIAL,
        WorkRequest.MIN_BACKOFF_MILLIS,
        TimeUnit.MILLISECONDS)
                .build()
            WorkManager.getInstance(context)
                .enqueueUniquePeriodicWork("helix_ota_check",
        ExistingPeriodicWorkPolicy.KEEP, request)
        }
    }
}
```

4.4 Dependencies

Dependency	Type	Purpose
`helix-ota-client-sdk` (AAR)	gomobile AAR	Server communication, download, verification
`androidx.work:work-runtime-ktx`	AndroidX	Periodic update checking
`androidx.core:core-ktx`	AndroidX	Core Kotlin extensions
`com.squareup.okhttp3:okhttp`	Third-party	HTTP client for non-SDK calls
`com.google.dagger:hilt-android`	Third-party	Dependency injection
Android `update_engine` AIDL	System	Binder IPC for payload application

4.5 Container Integration

The Android app is not containerized. However, the APK is included in the AOSP build system, which itself may run in containers during CI. The `Makefile` includes a target to pull the latest AAR from the `helix-ota-client-sdk` release artifacts and copy it into `app/libs/`.

4.6 Documentation Requirements

File	Required Sections
`README.md`	Build prerequisites (AOSP tree, SDK), APK signing, system app installation, WorkManager scheduling, update_engine integration guide
`CLAUDE.md`	Kotlin coding conventions, Android-specific test patterns (Robolectric vs instrumentation), AAR integration workflow
`AGENTS.md`	Module ownership, AAR version update protocol, release signing process

4.7 Test Coverage Requirements

Category	Target	Tool
Unit	85% line	JUnit 5, Mockito, Robolectric
Integration	update_engine mock	AndroidX Test, custom `IUpdateEngine` shadow
UI	Critical flows	Espresso
E2E	Full OTA cycle on Orange Pi 5 Max	Custom hardware test harness

5. helix-ota-dashboard

5.1 Overview

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-ota-dashboard`
Repository (GitLab)	`HelixDevelopment/helix-ota-dashboard`
Go module	N/A (TypeScript/React project)
Language	TypeScript 5.x, React 18+
Build tool	Vite 5.x
License	Apache 2.0
Visibility	Public

The Helix OTA Dashboard is a single-page application (SPA) providing administrators and operators with a web interface for managing artifacts, controlling rollouts, monitoring the device fleet, and viewing telemetry. It uses vasic-digital TypeScript submodules for API communication, authentication context, and UI components.

5.2 Package Structure

```
helix-ota-dashboard/
├── src/
│   ├── api/
│   │   ├── client.ts                  # API-Client-TS
wrapper
│   │   ├── artifacts.ts               # Artifact API
functions
│   │   ├── devices.ts                 # Device API
functions
│   │   ├── rollouts.ts                # Rollout API
functions
│   │   └── telemetry.ts              # Telemetry API
functions
│   ├── auth/
│   │   ├── AuthProvider.tsx           # Auth-Context-React
integration
│   │   ├── useAuth.ts                 # Auth hook
│   │   └── ProtectedRoute.tsx         # Route guard
│   ├── components/
│   │   ├── layout/
│   │   │   ├── AppShell.tsx           # Main layout using
UI-Components-React
│   │   │   ├── Sidebar.tsx
```

```
│   │   │   └── Header.tsx
│   │   ├── artifacts/
│   │   │   ├── ArtifactUpload.tsx      # Drag-and-drop
upload
│   │   │   ├── ArtifactList.tsx
│   │   │   └── ArtifactDetail.tsx
│   │   ├── rollouts/
│   │   │   ├── RolloutCreate.tsx       # Create rollout
wizard
│   │   │   ├── RolloutList.tsx
│   │   │   ├── RolloutDetail.tsx       # Progress
visualization
│   │   │   └── RolloutControls.tsx     # Pause/resume/halt
buttons
│   │   ├── devices/
│   │   │   ├── DeviceList.tsx
│   │   │   ├── DeviceDetail.tsx
│   │   │   └── DeviceFleetOverview.tsx # Fleet status cards
│   │   └── telemetry/
│   │       ├── TelemetryDashboard.tsx  # Charts and metrics
│   │       └── TelemetryTimeline.tsx
│   ├── hooks/
│   │   ├── useWebSocket.ts            # Real-time updates
via WS
│   │   └── usePolling.ts              # Fallback polling
│   ├── pages/
│   │   ├── Login.tsx
│   │   ├── Dashboard.tsx              # Overview page
│   │   ├── Artifacts.tsx
│   │   ├── Rollouts.tsx
│   │   ├── Devices.tsx
│   │   └── Telemetry.tsx
│   ├── types/
│   │   └── api.ts                     # Generated API types
│   ├── App.tsx
│   └── main.tsx
├── public/
├── package.json
├── tsconfig.json
├── vite.config.ts
├── Dockerfile                         # Nginx-based static
serving
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md
```

5.3 Core Types

```
// types/api.ts — Generated from OpenAPI spec
export interface Artifact {
```

```
  artifact_id: string;
  version: string;
  hardware_models: string[];
  os_type: string;
  size_bytes: number;
  sha256: string;
  validation_status: "passed" | "failed" | "pending";
  created_at: string;
}

export interface Rollout {
  id: string;
  artifact_id: string;
  status: "DRAFT" | "RUNNING" | "PAUSED" | "HALTED" |
        "COMPLETED";
  target_group: string;
  current_percentage: number;
  failure_threshold: number;
  stages: RolloutStage[];
  created_at: string;
  updated_at: string;
}

export interface Device {
  device_id: string;
  status: "registered" | "online" | "offline" |
        "decommissioned";
  hardware_model: string;
  firmware_version: string;
  os_type: string;
  last_seen: string;
}
```

5.4 Dependencies

vasic-digital TypeScript Submodules:

Submodule	Purpose
`API-Client-TS`	Type-safe HTTP client with interceptors for auth, retry, and error handling
`Auth-Context-React`	React context provider for JWT authentication, TOTP 2FA, token rotation
`UI-Components-React`	Shared UI component library (buttons, forms, tables, modals, charts)

External Dependencies: React 18, React Router 6, TanStack Query (data fetching), Recharts (charting), Zustand (state), Vite (build), Tailwind CSS (styling).

5.5 Container Integration

The dashboard is containerized as a multi-stage Docker build that produces an Nginx container serving static assets:

```
# Build stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

# Runtime stage
FROM nginx:1.25-alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
EXPOSE 80
```

The Nginx configuration proxies `/api/` and `/ws/` requests to the OTA server container, enabling the SPA to communicate with the backend without CORS issues.

5.6 Documentation Requirements

File	Required Sections
`README.md`	Quickstart (npm install, npm run dev), environment configuration, API proxy setup, vasic-digital TS submodule integration guide
`CLAUDE.md`	React/TypeScript conventions, component structure rules, API client usage patterns, state management rules
`AGENTS.md`	Page/component ownership, design system usage, i18n protocol

5.7 Test Coverage Requirements

Category	Target	Tool
Unit	85% line	Vitest, React Testing Library
Integration	API client	

Category	Target	Tool
		MSW (Mock Service Worker)
E2E	Critical flows	Playwright
Visual	Component regression	Chromatic (Storybook)

---

6. helix-update-engine

6.1 Overview

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-update-engine`
Repository (GitLab)	`HelixDevelopment/helix-update-engine`
Go module	`dev.helix.ota.updateengine`
Language	Go 1.22+
License	Apache 2.0
Visibility	Public

The Helix Update Engine is a universal update execution engine that provides an OS-agnostic abstraction over platform-specific update mechanisms. It defines the `OSAdapter` interface that each operating system must implement, and provides a state machine for the complete update lifecycle. The 1.0.0 release ships with the Android adapter; Linux and Windows adapters follow in 1.1.0 and 1.2.0 respectively.

6.2 Package Structure

```
helix-update-engine/
├── pkg/
│   ├── adapter/
│   │   ├── adapter.go                  # OSAdapter interface definition
│   │   └── adapter_test.go
│   ├── android/
│   │   ├── android.go                  # Android adapter: update_engine Binder IPC
│   │   ├── bootcontrol.go             # Boot control HAL interaction
│   │   ├── slotmanager.go             # A/B slot
```

```
management
│   │   └── android_test.go
│   ├── linux/
│   │   ├── linux.go                     # Linux adapter: A/
B partition + rpm-ostree
│   │   ├── partition.go                # Partition
management
│   │   ├── ostree.go                   # OSTree integration
│   │   └── linux_test.go
│   ├── windows/
│   │   ├── windows.go                   # Windows adapter:
MSI/MSIX installation
│   │   ├── msi.go                      # MSI package
handler
│   │   ├── service.go                  # Windows Service
integration
│   │   └── windows_test.go
│   ├── statemachine/
│   │   ├── machine.go                   # Update lifecycle
state machine
│   │   ├── states.go                    # State definitions
│   │   ├── events.go                    # Event definitions
│   │   ├── transitions.go              # Valid transition
map
│   │   └── statemachine_test.go
│   └── config/
│       ├── config.go                    # Engine
configuration
│       └── config_test.go
├── go.mod
├── go.sum
├── Makefile
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md
```

6.3 Core Types and Interfaces

```
// go.mod
module dev.helix.ota.updateengine

go 1.22

require (
    dev.helix.ota.deviceidentity v1.0.0
    github.com/google/uuid       v1.6.0
)

package adapter
```

```go
// OSAdapter defines the contract for OS-specific update
        logic.
// Each supported OS implements this interface.
type OSAdapter interface {
    // Name returns the adapter's OS identifier (e.g.,
        "android", "linux", "windows").
    Name() string

    // CheckReadiness verifies the device is in a state that
        can accept updates.
    // Checks: battery level, storage space, no active
        update, correct slot.
    CheckReadiness(ctx context.Context, req ReadinessCheck)
        (*ReadinessResult, error)

    // ApplyUpdate triggers the OS-specific update
        mechanism.
    ApplyUpdate(ctx context.Context, req ApplyRequest) error

    // VerifyUpdate confirms the update was applied
        correctly post-reboot.
    VerifyUpdate(ctx context.Context, req VerifyRequest)
        (*VerifyResult, error)

    // Rollback reverts to the previous system state.
    Rollback(ctx context.Context, req RollbackRequest) error

    // ReportStatus returns the current update status from
        the OS perspective.
    ReportStatus(ctx context.Context) (*UpdateStatus, error)

    // CancelUpdate aborts an in-progress update.
    CancelUpdate(ctx context.Context) error
}

// ReadinessCheck contains the criteria for determining
        update readiness.
type ReadinessCheck struct {
    MinBatteryPercent int   // Minimum battery level
        (default: 30)
    MinStorageBytes   int64 // Minimum free storage
    RequireACPower    bool  // Require AC power for update
    RequireWiFi       bool  // Require WiFi for download
}

// ReadinessResult indicates whether the device can accept
        an update.
type ReadinessResult struct {
    Ready  bool     `json:"ready"`
    Reason string   `json:"reason,omitempty"`
    Blocks []string `json:"blocks,omitempty"` // e.g.,
        ["battery_low", "storage_insufficient"]
}
```

```go
// ApplyRequest contains the information needed to apply an
        update.
type ApplyRequest struct {
    ArtifactPath string            `json:"artifact_path"`
    Metadata     map[string]string `json:"metadata"` // OS-
        specific metadata
}

// VerifyRequest contains the information needed to verify
        an applied update.
type VerifyRequest struct {
    ExpectedVersion string `json:"expected_version"`
}

// VerifyResult indicates whether the update was verified.
type VerifyResult struct {
    Verified       bool   `json:"verified"`
    CurrentVersion string `json:"current_version"`
    ActiveSlot     string `json:"active_slot"`
}

// RollbackRequest specifies the rollback target.
type RollbackRequest struct {
    TargetVersion string
        `json:"target_version,omitempty"` // Empty =
        previous slot
    Reason        string `json:"reason"`
}

// UpdateStatus represents the current update status.
type UpdateStatus struct {
    State           string `json:"state"` // IDLE,
        DOWNLOADING, VERIFYING, APPLYING, REBOOTING,
        SUCCEEDED, FAILED
    ProgressPercent int    `json:"progress_percent"`
    CurrentSlot     string `json:"current_slot"`
    CurrentVersion  string `json:"current_version"`
    ErrorCode       string `json:"error_code,omitempty"`
    ErrorMessage    string `json:"error_message,omitempty"`
}

package statemachine

// State represents a state in the update lifecycle.
type State string

const (
    StateIdle        State = "IDLE"
    StateChecking    State = "CHECKING"
    StateDownloading State = "DOWNLOADING"
    StateVerifying   State = "VERIFYING"
    StateApplying    State = "APPLYING"
    StateRebooting   State = "REBOOTING"
```

```go
    StateCommitting State = "COMMITTING"
    StateSucceeded  State = "SUCCEEDED"
    StateFailed     State = "FAILED"
    StateRollingBack State = "ROLLING_BACK"
)

// Event represents a trigger for a state transition.
type Event string

const (
    EventUpdateAvailable  Event = "UPDATE_AVAILABLE"
    EventDownloadComplete Event = "DOWNLOAD_COMPLETE"
    EventVerifyComplete   Event = "VERIFY_COMPLETE"
    EventApplyComplete    Event = "APPLY_COMPLETE"
    EventRebootComplete   Event = "REBOOT_COMPLETE"
    EventCommitComplete   Event = "COMMIT_COMPLETE"
    EventError            Event = "ERROR"
    EventRollbackTrigger  Event = "ROLLBACK_TRIGGER"
    EventRollbackComplete Event = "ROLLBACK_COMPLETE"
)

// Machine manages update lifecycle state transitions.
type Machine struct {
    current      State
    transitions  map[State]map[Event]State
    history      []Transition
    mu           sync.Mutex
}

// Transition attempts a state change. Returns error if the transition is invalid.
func (m *Machine) Transition(event Event) error { /* ... */ }

// CurrentState returns the current state.
func (m *Machine) CurrentState() State { /* ... */ }
```

6.4 Android Adapter

```go
package android

// AndroidAdapter implements OSAdapter for Android 15 A/B updates.
// It communicates with update_engine via the Binder IPC interface
// (proxied through the helix-ota-android Kotlin layer).
type AndroidAdapter struct {
    bootControl BootControl
    slotManager SlotManager
}

// BootControl abstracts the Android Boot Control HAL.
```

```go
type BootControl interface {
    GetCurrentSlot() (int, error)
    GetActiveSlotSuffix() (string, error)
    SetActiveBootSlot(slot int) error
    MarkBootSuccessful() error
}

// SlotManager manages A/B slot state.
type SlotManager interface {
    GetSlotInfo() (*SlotInfo, error)
    GetInactiveSlot() (int, error)
    IsSlotBootable(slot int) (bool, error)
}

type SlotInfo struct {
    ActiveSlot  int    `json:"active_slot"`
    SlotASuffix string `json:"slot_a_suffix"` // "_a"
    SlotBSuffix string `json:"slot_b_suffix"` // "_b"`
}
```

6.5 Linux Adapter (1.1.0)

```go
package linux

// LinuxAdapter implements OSAdapter for Linux A/B partition
        updates.
type LinuxAdapter struct {
    partitionMgr PartitionManager
    ostreeClient OSTreeClient
}

// PartitionManager manages A/B partition layout on Linux.
type PartitionManager interface {
    GetActivePartition() (string, error)
    GetInactivePartition() (string, error)
    WritePartition(device string, image io.Reader) error
    SwitchBootPartition(partition string) error
}

// OSTreeClient wraps the OSTree command-line interface.
type OSTreeClient interface {
    PullCommit(remote, ref string) error
    DeployCommit(ref string) error
    Rollback() error
}
```

6.6 Windows Adapter (1.2.0)

```go
package windows
```

```go
// WindowsAdapter implements OSAdapter for MSI/MSIX
        installation.
type WindowsAdapter struct {
    msiHandler  MSIHandler
    serviceCtrl ServiceController
}

// MSIHandler manages MSI/MSIX package installation.
type MSIHandler interface {
    Install(msiPath string, properties map[string]string)
        error
    Uninstall(productCode string) error
    GetInstalledVersion(productCode string) (string, error)
}

// ServiceController manages the Windows Service lifecycle.
type ServiceController interface {
    Install(serviceName, binaryPath string) error
    Start(serviceName string) error
    Stop(serviceName string) error
    QueryStatus(serviceName string) (string, error)
}
```

6.7 Dependencies

Submodule	Purpose
`dev.helix.ota.deviceidentity`	Device identity for readiness checks

6.8 Container Integration

The update engine is a library, not a standalone service. However, the Linux adapter may be used within a systemd-nspawn container or a Docker container that has access to the host's block devices (privileged mode for partition management).

6.9 Documentation Requirements

File	Required Sections
`README.md`	OSAdapter interface documentation, adapter implementation guide, state machine diagram, platform-specific adapter usage
`CLAUDE.md`	Go interface conventions, adapter registration pattern, test mocking strategies per adapter
`AGENTS.md`	

File	Required Sections
	Adapter ownership (one agent per OS adapter), cross-adapter interface stability protocol

6.10 Test Coverage Requirements

Category	Target	Notes
Unit	85% line	State machine, readiness checks, slot management
Mutation	75% score	Critical: state transition validity, rollback logic
Integration	Per-adapter	Android: mock Binder; Linux: mock partition; Windows: mock MSI
E2E	Real hardware	Android adapter on Orange Pi 5 Max

---

7. helix-artifact-validator

7.1 Overview

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-artifact-validator`
Repository (GitLab)	`HelixDevelopment/helix-artifact-validator`
Go module	`dev.helix.ota.artifactvalidator`
Language	Go 1.22+
License	Apache 2.0
Visibility	Public

The Helix Artifact Validator is a standalone validation library that performs multi-stage verification of OTA artifacts. It is used both server-side (during upload) and client-side (before installation) to ensure artifact integrity, authenticity, structure, and compatibility. The library supports streaming validation for large files, avoiding the need to load the entire artifact into memory.

7.2 Package Structure

```
helix-artifact-validator/
├── pkg/
│   ├── pipeline/
│   │   ├── pipeline.go                 # Multi-stage
validation pipeline
│   │   ├── stage.go                    # Stage interface
definition
│   │   └── pipeline_test.go
│   ├── structure/
│   │   ├── zip.go                      # ZIP structure
validation
│   │   ├── android.go                  # Android OTA ZIP
requirements
│   │   ├── linux.go                    # Linux rootfs
image requirements
│   │   ├── windows.go                  # Windows MSI/MSIX
requirements
│   │   └── structure_test.go
│   ├── hash/
│   │   ├── sha256.go                   # Streaming SHA-256
verification
│   │   └── hash_test.go
│   ├── signature/
│   │   ├── rsa.go                      # RSA-4096-PSS
signature verification
│   │   ├── ecdsa.go                    # ECDSA signature
(future)
│   │   ├── keyring.go                  # Public key
management
│   │   └── signature_test.go
│   ├── compatibility/
│   │   ├── checker.go                  # Device/model
compatibility check
│   │   ├── version.go                  # Semantic version
comparison
│   │   ├── model.go                    # Hardware model
registry
│   │   └── compatibility_test.go
│   ├── streaming/
│   │   ├── reader.go                   # Streaming reader
with hash computation
```

```
│   │   ├── writer.go                    # TeeWriter for
simultaneous write + validate
│   │   └── streaming_test.go
│   └── validator/
│       ├── validator.go                 # Main Validator
facade
│       └── validator_test.go
├── go.mod
├── go.sum
├── Makefile
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md
```

7.3 Core Types and Interfaces

```go
// go.mod
module dev.helix.ota.artifactvalidator

go 1.22

require (
    golang.org/x/crypto   v0.18.0
    github.com/google/uuid v1.6.0
    archive/zip            // stdlib
)

package pipeline

// Stage defines a single validation step in the pipeline.
type Stage interface {
    // Name returns the stage identifier (e.g., "hash",
        "signature", "structure").
    Name() string

    // Validate executes the validation step. Returns a
        StageResult.
    // If the stage fails, the pipeline may short-circuit
        (configurable).
    Validate(ctx context.Context, input *ValidationInput)
        (*StageResult, error)
}

// StageResult captures the outcome of a single validation
        stage.
type StageResult struct {
    Name    string `json:"name"`
    Passed  bool   `json:"passed"`
    Message string `json:"message"`
    Duration time.Duration `json:"duration"`
}
```

```go
// ValidationResult captures the combined outcome of all
        stages.
type ValidationResult struct {
    Valid   bool          `json:"valid"`
    Errors  []string      `json:"errors,omitempty"`
    Stages  []StageResult `json:"stages"`
    TotalDuration time.Duration `json:"total_duration"`
}

// ValidationInput provides the data needed for validation.
type ValidationInput struct {
    FilePath         string            // Path to the
        artifact file on disk
    Reader           io.Reader         // Alternative:
        streaming reader
    ExpectedSHA256   string            // Server-provided
        expected hash
    Signature        []byte            // RSA signature
        bytes
    PublicKey        *rsa.PublicKey    // Public key for
        signature verification
    TargetModel      string            // e.g.,
        "rk3588_opi5max"
    TargetVersion    string            // e.g., "15.0.1"
    MinSourceVersion string            // Minimum source
        version for update
    OSType           string            // "android",
        "linux", "windows"
    MaxFileSize      int64             // Maximum allowed
        file size
}

// Pipeline orchestrates a sequence of validation stages.
type Pipeline struct {
    stages       []Stage
    shortCircuit bool // If true, stop on first failure
}

// Validate runs all stages and returns the combined result.
func (p *Pipeline) Validate(ctx context.Context, input
        *ValidationInput) (*ValidationResult, error) {
    result := &ValidationResult{}
    for _, stage := range p.stages {
        stageResult, err := stage.Validate(ctx, input)
        if err != nil {
            return nil, fmt.Errorf("stage %s: %w",
        stage.Name(), err)
        }
        result.Stages = append(result.Stages, *stageResult)
        if !stageResult.Passed {
            result.Valid = false
            result.Errors = append(result.Errors,
        stageResult.Message)
```

```go
            if p.shortCircuit {
                return result, nil
            }
        }
    }
    result.Valid = len(result.Errors) == 0
    return result, nil
}

// NewDefaultPipeline creates the standard four-stage
        validation pipeline:
// Hash → Signature → Structure → Compatibility
func NewDefaultPipeline() *Pipeline {
    return &Pipeline{
        stages: []Stage{
            &hash.SHA256Stage{},
            &signature.RSAPSSStage{},
            &structure.ZIPStructureStage{},
            &compatibility.CheckerStage{},
        },
        shortCircuit: true,
    }
}
```

7.4 Streaming Validation

```go
package streaming

// ValidatingReader wraps an io.Reader and computes SHA-256
        while reading.
// This enables validation of large artifacts (>2GB) without
        loading them into memory.
type ValidatingReader struct {
    reader    io.Reader
    hash      hash.Hash
    bytesRead int64
}

// NewValidatingReader creates a reader that computes
        SHA-256 on the fly.
func NewValidatingReader(r io.Reader) *ValidatingReader {
    return &ValidatingReader{
        reader: r,
        hash:   sha256.New(),
    }
}

// Read reads data and simultaneously updates the hash.
func (vr *ValidatingReader) Read(p []byte) (int, error) {
    n, err := vr.reader.Read(p)
    if n > 0 {
        vr.hash.Write(p[:n])
```

```go
        vr.bytesRead += int64(n)
    }
    return n, err
}

// Sum returns the SHA-256 hash of all data read so far.
func (vr *ValidatingReader) Sum() string {
    return hex.EncodeToString(vr.hash.Sum(nil))
}

// BytesRead returns the total bytes read.
func (vr *ValidatingReader) BytesRead() int64 {
    return vr.bytesRead
}

// ValidatingWriter wraps an io.Writer and computes SHA-256
        while writing.
// Used during upload to validate as the file is streamed to
        storage.
type ValidatingWriter struct {
    writer    io.Writer
    hash      hash.Hash
    bytesRead int64
}

// Write writes data and simultaneously updates the hash.
func (vw *ValidatingWriter) Write(p []byte) (int, error) {
    n, err := vw.writer.Write(p)
    if n > 0 {
        vw.hash.Write(p[:n])
        vw.bytesRead += int64(n)
    }
    return n, err
}

// Sum returns the SHA-256 hash of all data written.
func (vw *ValidatingWriter) Sum() string {
    return hex.EncodeToString(vw.hash.Sum(nil))
}
```

7.5 Dependencies

No Helix OTA internal submodule dependencies. External: `golang.org/x/crypto` (for RSA-PSS), Go stdlib `archive/zip`, `crypto/sha256`, `crypto/rsa`.

7.6 Container Integration

The artifact validator is a library, not a service. It is used within the server's validation worker pool (running inside the server container) and within client-side SDK processes. No independent containerization needed.

7.7 Documentation Requirements

File	Required Sections
`README.md`	Validation pipeline overview, stage documentation, streaming API guide, custom stage implementation, Android/Linux/Windows structure requirements
`CLAUDE.md`	Pipeline extension patterns, streaming gotchas (must read to EOF for valid hash), concurrent validation with worker pools
`AGENTS.md`	Stage ownership (one agent per stage), pipeline configuration protocol

7.8 Test Coverage Requirements

Category	Target	Notes
Unit	85% line	Each stage in isolation, hash/signature positive and negative cases
Mutation	75% score	Critical: hash comparison logic, signature verification, version comparison
Integration	Full pipeline	Valid artifact passes all stages; corrupted/tampered artifact fails at correct stage
Performance	2GB artifact < 30s	Streaming validation benchmark

8. helix-rollout-engine

8.1 Overview

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-rollout-engine`
Repository (GitLab)	`HelixDevelopment/helix-rollout-engine`
Go module	`dev.helix.ota.rolloutengine`
Language	Go 1.22+
License	Apache 2.0
Visibility	Public

The Helix Rollout Engine is a phased rollout decision engine that controls how updates are progressively delivered to device fleets. It provides deterministic percentage-based device selection, configurable rollout strategies (instant, canary, gradual), auto-rollback with configurable failure thresholds, and is completely decoupled from any specific storage backend through repository interfaces.

8.2 Package Structure

```
helix-rollout-engine/
├── pkg/
│   ├── engine/
│   │   ├── engine.go                    # RolloutEngine
main interface
│   │   ├── decision.go                  # Decision-making
logic
│   │   └── engine_test.go
│   ├── selector/
│   │   ├── selector.go                  # DeviceSelector
with deterministic hashing
│   │   ├── hash.go                      # FNV-32 and
SHA-256 hash functions
│   │   └── selector_test.go
│   ├── strategy/
│   │   ├── strategy.go                  # RolloutStrategy
interface
│   │   ├── instant.go                   # Instant rollout
(0→100%)
│   │   ├── canary.go                    # Canary rollout
(1%→5%→10%→25%→50%→100%)
│   │   ├── gradual.go                   # Gradual rollout
(5%→10%→30%→50%→100%)
│   │   ├── custom.go                    # User-defined
```

```
stages
│   │   └── strategy_test.go
│   ├── rollback/
│   │   ├── auto.go                     # Auto-rollback
with failure threshold
│   │   ├── circuitbreaker.go           # Circuit breaker
for rollback loops
│   │   └── rollback_test.go
│   ├── model/
│   │   ├── rollout.go                  # Rollout domain
model
│   │   ├── stage.go                    # Rollout stage
definition
│   │   ├── status.go                   # Status enum
│   │   └── errors.go                   # Domain errors
│   ├── repository/
│   │   └── repository.go              # Storage backend
interface (decoupled)
│   └── config/
│       ├── config.go                   # Engine
configuration
│       └── config_test.go
├── go.mod
├── go.sum
├── Makefile
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md
```

8.3 Core Types and Interfaces

```
// go.mod
module dev.helix.ota.rolloutengine

go 1.22

require (
    github.com/google/uuid v1.6.0
)
```

```go
package engine

// RolloutEngine controls the phased delivery of updates to
        device fleets.
type RolloutEngine interface {
    // CreateRollout initializes a new rollout with the
        given strategy.
    CreateRollout(ctx context.Context, req
        CreateRolloutRequest) (*model.Rollout, error)
```

```go
    // Evaluate evaluates a rollout and decides whether to
        advance, pause, or halt.
    Evaluate(ctx context.Context, rolloutID string)
        (*Decision, error)

    // Advance moves the rollout to the next stage.
    Advance(ctx context.Context, rolloutID string)
        (*model.Rollout, error)

    // Pause halts the rollout at the current stage.
    Pause(ctx context.Context, rolloutID string) error

    // Resume continues a paused rollout.
    Resume(ctx context.Context, rolloutID string) error

    // Halt permanently stops a rollout. Optionally triggers
        auto-rollback.
    Halt(ctx context.Context, rolloutID string, reason
        string) error
}

// Decision represents the engine's evaluation result for a
        rollout.
type Decision struct {
    Action       DecisionAction `json:"action"` // ADVANCE,
        HOLD, PAUSE, HALT, ROLLBACK
    Reason       string         `json:"reason"`
    CurrentStage int            `json:"current_stage"`
    NextStage    int
        `json:"next_stage,omitempty"`
    Metrics      *HealthMetrics `json:"metrics"`
}

type DecisionAction string

const (
    ActionAdvance  DecisionAction = "ADVANCE"
    ActionHold     DecisionAction = "HOLD"
    ActionPause    DecisionAction = "PAUSE"
    ActionHalt     DecisionAction = "HALT"
    ActionRollback DecisionAction = "ROLLBACK"
)

// HealthMetrics contains the health signals used for
        rollout decisions.
type HealthMetrics struct {
    TotalDevices    int     `json:"total_devices"`
    SucceededCount  int     `json:"succeeded_count"`
    FailedCount     int     `json:"failed_count"`
    InProgressCount int     `json:"in_progress_count"`
    FailureRate     float64 `json:"failure_rate"`
    SuccessRate     float64 `json:"success_rate"`
}
```

```go
// CreateRolloutRequest contains the parameters for creating
        a new rollout.
type CreateRolloutRequest struct {
    ArtifactID        string          `json:"artifact_id"`
    DeviceGroup       string          `json:"device_group"`
    Strategy          RolloutStrategy `json:"strategy"`
    FailureThreshold  float64
        `json:"failure_threshold"` // 0.0–1.0, default 0.05
    AutoRollback      bool            `json:"auto_rollback"`
    Mandatory         bool            `json:"mandatory"`
    Deadline          *time.Time
        `json:"deadline,omitempty"`
}

package strategy

// RolloutStrategy defines how a rollout progresses through
        stages.
type RolloutStrategy interface {
    // Name returns the strategy identifier.
    Name() string

    // Stages returns the ordered list of rollout stages.
    Stages() []model.RolloutStage

    // DefaultThreshold returns the default failure
        threshold for this strategy.
    DefaultThreshold() float64
}

// InstantStrategy delivers the update to 100% of devices
        immediately.
type InstantStrategy struct{}

func (s *InstantStrategy) Stages() []model.RolloutStage {
    return []model.RolloutStage{
        {Percentage: 100, MinDwellDuration: 0},
    }
}

// CanaryStrategy delivers to a small canary group first,
        then expands.
type CanaryStrategy struct{}

func (s *CanaryStrategy) Stages() []model.RolloutStage {
    return []model.RolloutStage{
        {Percentage: 1, MinDwellDuration: 1 * time.Hour},
        {Percentage: 5, MinDwellDuration: 2 * time.Hour},
        {Percentage: 10, MinDwellDuration: 4 * time.Hour},
        {Percentage: 25, MinDwellDuration: 8 * time.Hour},
        {Percentage: 50, MinDwellDuration: 12 * time.Hour},
        {Percentage: 100, MinDwellDuration: 0},
```

```go
    }
}

// GradualStrategy delivers in standard progressive stages.
type GradualStrategy struct{}

func (s *GradualStrategy) Stages() []model.RolloutStage {
    return []model.RolloutStage{
        {Percentage: 5, MinDwellDuration: 30 * time.Minute},
        {Percentage: 10, MinDwellDuration: 1 * time.Hour},
        {Percentage: 30, MinDwellDuration: 4 * time.Hour},
        {Percentage: 50, MinDwellDuration: 12 * time.Hour},
        {Percentage: 100, MinDwellDuration: 0},
    }
}

package selector

// DeviceSelector determines which devices receive the update at each stage.
type DeviceSelector struct{}

// SelectForPercentage returns devices that fall within the rollout percentage.
// Uses deterministic hashing: cohort = SHA256(rolloutID:deviceID) mod 100
// This ensures:
//   - Same device always maps to same cohort for a given rollout
//   - Increasing percentage always includes previously selected devices (monotonic)
//   - Uniform distribution across the fleet
func (s *DeviceSelector) SelectForPercentage(
    devices []DeviceCandidate,
    rolloutID string,
    percentage float64,
) []DeviceCandidate { /* ... */ }

package repository

// RolloutRepository defines the storage backend interface.
// This is decoupled from any specific database — the consuming application
// provides the implementation (PostgreSQL, SQLite, in-memory for tests).
type RolloutRepository interface {
    Create(ctx context.Context, rollout *model.Rollout) error
    GetByID(ctx context.Context, id string) (*model.Rollout, error)
    Update(ctx context.Context, rollout *model.Rollout) error
    ListByStatus(ctx context.Context, status model.RolloutStatus) ([]*model.Rollout, error)
```

```go
}

// TelemetryRepository defines the telemetry backend
        interface.
type TelemetryRepository interface {
    GetRolloutStats(ctx context.Context, rolloutID string,
        since *time.Time) (*RolloutStats, error)
}

// RolloutStats contains aggregate statistics for a rollout.
type RolloutStats struct {
    CompletedCount      int
    FailedCount         int
    InProgressCount     int
    AvgDownloadDuration time.Duration
    AvgInstallDuration  time.Duration
}
```

8.4 Auto-Rollback with Circuit Breaker

```go
package rollback

// AutoRollback evaluates failure metrics and triggers
        rollback if thresholds are exceeded.
type AutoRollback struct {
    threshold      float64 // Maximum failure rate (0.0–1.0)
    circuitBreaker *CircuitBreaker
}

// CircuitBreaker prevents rollback loops by limiting
        rollback attempts.
type CircuitBreaker struct {
    maxAttempts
        int           // Maximum rollback attempts per
        rollout (default: 3)
    window       time.Duration // Time window for counting
        attempts (default: 24h)
    attemptCount  int
    windowStart   time.Time
}

// ShouldRollback returns true if the failure rate exceeds
        the threshold
// and the circuit breaker has not been tripped.
func (ar *AutoRollback) ShouldRollback(metrics
        *engine.HealthMetrics) bool {
    if metrics.FailureRate > ar.threshold {
        return ar.circuitBreaker.Allow()
    }
    return false
}
```

8.5 Dependencies

No Helix OTA internal submodule dependencies. The rollout engine is fully self-contained with only external (stdlib + uuid) dependencies. Storage backends are injected via interfaces.

8.6 Container Integration

The rollout engine is a library embedded within the OTA server container. It does not run as an independent container. The server's background task scheduler invokes `engine.Evaluate()` on a configurable interval (default: 15 minutes).

8.7 Documentation Requirements

File	Required Sections
`README.md`	Strategy comparison table, deterministic cohort algorithm explanation, auto-rollback configuration, repository interface guide, circuit breaker tuning
`CLAUDE.md`	Strategy extension pattern, deterministic hash invariants (must be stable across server restarts), test patterns for concurrent rollout evaluation
`AGENTS.md`	Strategy ownership, repository interface stability guarantees, evaluation scheduling protocol

8.8 Test Coverage Requirements

Category	Target	Notes
Unit	85% line, 120+ tests	All strategies, selector, decision engine, circuit breaker, rollback logic
Mutation	75% score	Critical: hash-to-cohort mapping, failure rate calculation, stage progression
Integration	PostgreSQL + testcontainers	Full create→evaluate→advance→complete lifecycle
Concurrency	1000 devices	Parallel device assignment with Redis lock

---

9. helix-device-identity

9.1 Overview

Property	Value
Repository (GitHub)	`HelixDevelopment/helix-device-identity`
Repository (GitLab)	`HelixDevelopment/helix-device-identity`
Go module	`dev.helix.ota.deviceidentity`
Language	Go 1.22+
License	Apache 2.0
Visibility	Public

The Helix Device Identity library provides hardware-bound device ID generation, mTLS certificate enrollment and rotation, certificate revocation list management, and platform-specific identity providers. It ensures that each device has a cryptographically verifiable identity that is bound to the device's hardware and cannot be cloned or stolen.

9.2 Package Structure

```
helix-device-identity/
├── pkg/
│   ├── identity/
│   │   ├── identity.go                 # DeviceIdentity
main type
│   │   ├── generator.go                # Hardware-bound ID
generation
│   │   └── identity_test.go
│   ├── enrollment/
│   │   ├── enrollment.go               # Certificate
enrollment interface
│   │   ├── csr.go                      # CSR generation
│   │   ├── enrollment_test.go
│   ├── rotation/
│   │   ├── rotation.go                 # Certificate
rotation logic
│   │   ├── scheduler.go                # Rotation
scheduling
│   │   └── rotation_test.go
│   ├── revocation/
│   │   ├── crl.go                      # Certificate
Revocation List management
│   │   ├── ocsp.go                     # OCSP responder
client
```

```
│   │   ├── store.go                    # Revocation store
interface
│   │   └── revocation_test.go
│   ├── provider/
│   │   ├── provider.go                 # IdentityProvider
interface
│   │   ├── android.go                  # Android Keystore
provider
│   │   ├── linux.go                    # Linux TPM/TEE
provider
│   │   ├── windows.go                  # Windows TPM
provider
│   │   └── provider_test.go
│   ├── fingerprint/
│   │   ├── fingerprint.go             # Hardware
fingerprint computation
│   │   ├── android.go                  # Android hardware
properties
│   │   ├── linux.go                    # Linux DMI/SMBIOS
properties
│   │   └── fingerprint_test.go
│   └── ca/
│       ├── ca.go                       # Certificate
Authority client
│       ├── signer.go                   # Certificate
signing
│       └── ca_test.go
├── go.mod
├── go.sum
├── Makefile
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md
```

9.3 Core Types and Interfaces

```
// go.mod
module dev.helix.ota.deviceidentity

go 1.22

require (
    golang.org/x/crypto   v0.18.0
    github.com/google/uuid v1.6.0
)

package identity

// DeviceIdentity represents a device's cryptographic
        identity.
```

```go
type DeviceIdentity struct {
    DeviceID    string    `json:"device_id"`
    Fingerprint string    `json:"fingerprint"` // Hardware-
        bound hash
    Certificate []byte    `json:"certificate"` // PEM-
        encoded X.509 certificate
    PublicKey   []byte    `json:"public_key"`  // PEM-
        encoded public key
    IssuedAt    time.Time `json:"issued_at"`
    ExpiresAt   time.Time `json:"expires_at"`
    Provider    string    `json:"provider"`    //
        "android_keystore", "linux_tpm", "windows_tpm"
}

// Generator creates hardware-bound device identities.
type Generator interface {
    // Generate creates a new device identity bound to the
        hardware.
    // The private key is generated inside the platform's
        secure element
    // and is non-exportable.
    Generate(ctx context.Context, hwProps
        HardwareProperties) (*DeviceIdentity, error)

    // Verify verifies that a device identity matches the
        current hardware.
    Verify(ctx context.Context, identity *DeviceIdentity)
        (bool, error)
}

// HardwareProperties contains the hardware attributes used
        for identity binding.
type HardwareProperties struct {
    SerialNumber     string `json:"serial_number"`
    CPUID            string `json:"cpu_id"`
    MACAddress       string `json:"mac_address"`
    BoardRevision    string `json:"board_revision"`
    SecureBootState  bool   `json:"secure_boot_state"`
    BootloaderVersion string `json:"bootloader_version"`
}

package enrollment

// Enroller manages the certificate enrollment lifecycle.
type Enroller interface {
    // GenerateCSR creates a Certificate Signing Request
        using the device's
    // hardware-bound key pair. The private key never leaves
        the secure element.
    GenerateCSR(ctx context.Context, identity
        *DeviceIdentity) ([]byte, error)

    // SubmitCSR submits the CSR to the Helix OTA
        Certificate Authority.
```

```go
    SubmitCSR(ctx context.Context, csrPEM []byte)
        (*x509.Certificate, error)

    // InstallCertificate installs the signed certificate on
        the device.
    InstallCertificate(ctx context.Context, cert
        *x509.Certificate) error
}

package rotation

// RotationManager manages certificate lifecycle and
        rotation.
type RotationManager struct {
    enroller     enrollment.Enroller
    revocation   revocation.Revoker
    renewalDays  int // Days before expiry to trigger
        renewal (default: 60)
}

// RotateIfNeeded checks if the device certificate needs
        rotation and performs it.
func (rm *RotationManager) RotateIfNeeded(ctx
        context.Context, identity *DeviceIdentity)
        (*DeviceIdentity, error) {
    daysUntilExpiry :=
        time.Until(identity.ExpiresAt).Hours() / 24
    if daysUntilExpiry > float64(rm.renewalDays) {
        return identity, nil // No rotation needed
    }

    // Generate new CSR with existing valid certificate for
        authentication
    csr, err := rm.enroller.GenerateCSR(ctx, identity)
    if err != nil {
        return nil, fmt.Errorf("generate CSR: %w", err)
    }

    // Submit CSR using current valid certificate as auth
    newCert, err := rm.enroller.SubmitCSR(ctx, csr)
    if err != nil {
        return nil, fmt.Errorf("submit CSR: %w", err)
    }

    // Install new certificate
    if err := rm.enroller.InstallCertificate(ctx, newCert);
        err != nil {
        return nil, fmt.Errorf("install certificate: %w",
        err)
    }

    // Revoke old certificate (grace period: 24 hours)
    go rm.revocation.ScheduleRevocation(context.Background(),
        identity.Certificate, 24*time.Hour)
```

```go
	return &DeviceIdentity{
		DeviceID:    identity.DeviceID,
		Fingerprint: identity.Fingerprint,
		Certificate: encodePEM(newCert),
		PublicKey:   identity.PublicKey,
		IssuedAt:    time.Now(),
		ExpiresAt:   newCert.NotAfter,
		Provider:    identity.Provider,
	}, nil
}

package revocation

// Revoker manages certificate revocation.
type Revoker interface {
	// Revoke marks a certificate as revoked.
	Revoke(ctx context.Context, certPEM []byte, reason
		string) error

	// ScheduleRevocation revokes a certificate after a
		delay (for overlap periods).
	ScheduleRevocation(ctx context.Context, certPEM []byte,
		delay time.Duration) error

	// IsRevoked checks if a certificate is in the
		revocation list.
	IsRevoked(ctx context.Context, serialNumber string)
		(bool, error)

	// GetCRL returns the current Certificate Revocation
		List.
	GetCRL(ctx context.Context) ([]byte, error)
}

// Store defines the storage backend for revocation data.
type Store interface {
	AddRevocation(ctx context.Context, serialNumber string,
		reason string, revokedAt time.Time) error
	IsRevoked(ctx context.Context, serialNumber string)
		(bool, error)
	ListRevocations(ctx context.Context, since *time.Time)
		([]RevocationEntry, error)
}

package provider

// IdentityProvider abstracts platform-specific identity
		mechanisms.
type IdentityProvider interface {
	// Name returns the provider identifier (e.g.,
		"android_keystore").
	Name() string
```

```go
    // GenerateKeyPair generates a hardware-bound key pair.
    // The private key is non-exportable and stored in the
        secure element.
    GenerateKeyPair(ctx context.Context) (*KeyPair, error)

    // Sign signs data with the hardware-bound private key.
    Sign(ctx context.Context, digest []byte) ([]byte, error)

    // GetAttestation returns a hardware attestation
        certificate chain.
    GetAttestation(ctx context.Context)
        ([]*x509.Certificate, error)
}

// KeyPair represents a hardware-bound key pair.
type KeyPair struct {
    PublicKey  []byte `json:"public_key"`  // PEM-encoded
    KeyID      string `json:"key_id"`      // Platform-
        specific key identifier
    Provider   string `json:"provider"`
}
```

9.4 Android Keystore Provider

```go
package provider

// AndroidKeyStoreProvider implements IdentityProvider for
        Android devices.
// It uses Android Keystore (hardware-backed on RK3588) for
        key generation
// and Key Attestation for verifying key hardware binding.
//
// This provider is called via gomobile from the helix-ota-
        android app,
// which has access to the Android Keystore system APIs.
type AndroidKeyStoreProvider struct {
    alias string // Keystore entry alias
}

func (p *AndroidKeyStoreProvider) Name() string { return
        "android_keystore" }
```

9.5 Dependencies

No Helix OTA internal submodule dependencies. External: `golang.org/x/crypto` (for key generation and certificate handling), Go stdlib `crypto/x509`, `crypto/rsa`, `crypto/tls`.

9.6 Container Integration

The device identity library is used client-side (within the Android app, future Linux/Windows clients) and server-side (for certificate verification during device authentication). On the server side, it runs within the OTA server container for verifying device certificates and checking revocation status.

9.7 Documentation Requirements

File	Required Sections
`README.md`	Identity enrollment flow diagram, rotation schedule, CRL distribution, platform-specific provider setup (Android Keystore, Linux TPM, Windows TPM), hardware fingerprint composition
`CLAUDE.md`	Provider implementation guide, certificate parsing gotchas, Android Keystore constraints (no direct Go access, must go through Kotlin/gomobile), CRL update protocol
`AGENTS.md`	Provider ownership (one agent per platform), certificate lifecycle protocol, CA interaction specification

9.8 Test Coverage Requirements

Category	Target	Notes
Unit	85% line	Identity generation, CSR generation, rotation logic, CRL management
Mutation	75% score	Critical: fingerprint computation, certificate expiry check, revocation verification
Integration	Full enrollment flow	Generate → CSR → Sign → Install → Verify
Security		

Category	Target	Notes
	Certificate forgery test	Verify that forged certificates are rejected

10. Cross-Cutting Integration Map

10.1 Submodule Dependency Graph

```
                        ┌─────────────────────┐
                        │  helix-ota-server    │
                        │  (dev.helix.ota.server) │
                        └─────────────────────┘
                           │  │  │  │
                           │  │  │  │
             ┌─────────────┘  │  │  └──────────────┐
             ▼                │  ▼                 ▼
                             │
     ┌──────────────────┐   │  ┌──────────────────┐
  ┌──────────────────┐
     │ helix-rollout-    │      │ │ helix-artifact-   │ │ helix-
device-     │
     │ engine            │      │ │ validator         │ │
identity          │
     │ (dev.helix.ota.   │      │ │ (dev.helix.ota.   │ │
(dev.helix.ota.   │
     │  rolloutengine)   │      │ │  artifactvalidator)│ │
deviceidentity) │
     └──────────────────┘   │  └──────────────────┘

  └──────────────────┘
                             │
                             ▼
                   ┌──────────────────┐
                   │ helix-update-     │
                   │ engine            │
                   │ (dev.helix.ota.   │
                   │  updateengine)    │
                   └──────────────────┘
                            │
             ┌──────────────┴────────────┐
             ▼                            ▼
     ┌──────────────────┐   ┌──────────────────┐
     │ helix-ota-        │   │ helix-ota-        │
     │ client-sdk        │   │ android           │
     │ (dev.helix.ota.   │   │ (Kotlin/Android)  │
     │  clientsdk)       │   └──────────────────┘
     └──────────────────┘
            │ uses
            ▼
```

```
┌──────────────────┐     ┌──────────────────┐
│ helix-update-    │     │ helix-artifact-  │
│ engine           │     │ validator        │
│ (statemachine,   │     │ (hash, signature,│
│  adapter)        │     │  structure)      │
└──────────────────┘     └──────────────────┘

┌──────────────────┐
│ helix-ota-       │
│ dashboard        │
│ (React/TypeScript)│
└──────────────────┘
     Uses: API-Client-TS, Auth-Context-React, UI-Components-
React
```

10.2 Dependency Summary Table

Submodule	Depends on Helix OTA Submodules	Depends on vasic-digital Submodules
`helix-ota-server`	rolloutengine, artifactvalidator, deviceidentity, updateengine	auth, database, cache, observability, security, middleware, config, eventbus, storage, ratelimiter, concurrency, recovery, backgroundtasks
`helix-ota-client-sdk`	updateengine, artifactvalidator, deviceidentity	None
`helix-ota-android`	client-sdk (AAR)	None
`helix-ota-dashboard`	None (uses server REST API)	API-Client-TS, Auth-Context-React, UI-Components-React
`helix-update-engine`	deviceidentity	None
`helix-artifact-validator`	None	None
`helix-rollout-engine`	None	None
`helix-device-identity`	None	None

---

11. Container Integration Strategy

11.1 vasic-digital/containers Submodule

All Helix OTA containers are defined within the `vasic-digital/containers` submodule, which provides the shared Docker Compose and Kubernetes manifests. Each new submodule contributes:

Submodule	Container Contribution	Docker Image
`helix-ota-server`	Server Dockerfile + compose service	`helix-ota/server:latest`
`helix-ota-dashboard`	Dashboard Dockerfile (Nginx) + compose service	`helix-ota/dashboard:latest`
`helix-ota-client-sdk`	No container (library)	N/A
`helix-ota-android`	No container (APK)	N/A
`helix-update-engine`	No container (library)	N/A
`helix-artifact-validator`	No container (library)	N/A
`helix-rollout-engine`	No container (library)	N/A
`helix-device-identity`	No container (library)	N/A

11.2 Docker Compose Integration

The `docker-compose.yml` in the `containers` submodule is extended to include:

```yaml
services:
  helix-ota-server:
    build:
      context: ../helix-ota-server
      dockerfile: Dockerfile
    ports:
      - "8080:8080"   # HTTP (dev only)
      - "8443:8443"   # HTTPS
    environment:
```

```yaml
      - DATABASE_URL=postgres://helix:helix@postgres:5432/
        helix_ota
      - REDIS_URL=redis://redis:6379
      - MINIO_ENDPOINT=minio:9000
      - MINIO_ACCESS_KEY=minioadmin
      - MINIO_SECRET_KEY=minioadmin
      - TLS_CERT_PATH=/certs/server.crt
      - TLS_KEY_PATH=/certs/server.key
      - CA_CERT_PATH=/certs/ca.crt
    volumes:
      - ./certs:/certs:ro
    depends_on:
      - postgres
      - redis
      - minio

  helix-ota-dashboard:
    build:
      context: ../helix-ota-dashboard
      dockerfile: Dockerfile
    ports:
      - "3000:80"
    depends_on:
      - helix-ota-server

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: helix_ota
      POSTGRES_USER: helix
      POSTGRES_PASSWORD: helix
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data

  minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    volumes:
      - minio_data:/data
    ports:
      - "9000:9000"
      - "9001:9001"

volumes:
```

```yaml
  postgres_data:
  redis_data:
  minio_data:
```

11.3 Kubernetes Manifests

Production Kubernetes manifests are also maintained in the `containers` submodule:

- `helm/helix-ota-server/` — Helm chart for the OTA server
- `helm/helix-ota-dashboard/` — Helm chart for the dashboard
- `k8s/postgres/` — StatefulSet for PostgreSQL
- `k8s/redis/` — StatefulSet for Redis
- `k8s/minio/` — StatefulSet for MinIO
- `k8s/secrets/` — TLS certificates and signing key references

---

12. Release Checklist Template

The following checklist must be completed for every submodule release. No release is published until all items are checked.

12.1 Pre-Release

- [ ] All code merged to `main` via approved pull request
- [ ] All CI checks pass (lint, test, mutation, build)
- [ ] Line coverage ≥ 85% confirmed by CI
- [ ] Mutation score ≥ 75% confirmed by CI
- [ ] All integration tests pass against testcontainers
- [ ] No critical or high-severity open issues
- [ ] `CHANGELOG.md` updated with version, date, and changes
- [ ] `go.mod` dependencies reviewed and pinned to correct versions
- [ ]

README.md, CLAUDE.md, AGENTS.md reviewed for accuracy

12.2 Build & Publish

- [ ] Git tag created: `v1.0.0-mvp` (or appropriate version)
- [ ] Tag pushed to GitHub: `git push origin v1.0.0-mvp`
- [ ] GitLab mirror confirmed: tag appears in GitLab repository
- [ ] GitHub Release created with changelog contents
- [ ] Container image built and pushed: `helix-ota/<name>:v1.0.0-mvp` (if applicable)
- [ ] AAR artifact built and attached to GitHub Release (for `helix-ota-client-sdk`)
- [ ] APK artifact built and attached to GitHub Release (for `helix-ota-android`)

12.3 Post-Release

- [ ] Dependent submodules updated to reference new version in `go.mod` / `package.json`
- [ ] Integration test suite passes with new version
- [ ] Docker Compose `up` verified with new container images
- [ ] Helm chart `values.yaml` updated with new image tag
- [ ] Release announced in project communication channel
- [ ] Documentation site updated (if applicable)

12.4 Emergency Rollback Procedure

If a release introduces a critical defect:

1. Revert the merge commit on `main`
2. Tag a hotfix: `v1.0.0-mvp.1`
3. Rebuild and republish container image

4. Update dependent submodule references
5. Document the rollback in `CHANGELOG.md`

Appendix A — vasic-digital Submodule Dependency Matrix

This matrix shows which vasic-digital infrastructure submodules are used by which Helix OTA submodules:

vasic-digital Submodule	ota-server	client-sdk	android	dashboard	update-engine	artifa valida
auth	✅					
database	✅					
cache	✅					
observability	✅					
security	✅					
middleware	✅					
config	✅					
eventbus	✅					
storage	✅					
ratelimiter	✅					
concurrency	✅					
recovery	✅					
backgroundtasks	✅					
API-Client-TS				✅		
Auth-Context-React				✅		
UI-Components-React				✅		
containers	✅			✅		

Appendix B — Documentation Requirements per HelixConstitution

Per HelixConstitution §3.1 (Nano-Detail Documentation), every submodule must produce documentation at specification grade. The following table defines the required documentation artifacts for each submodule:

Submodule	README.md	CLAUDE.md	AGENTS.md	CONTRIBUTIN
helix-ota-server	✅	✅	✅	✅
helix-ota-client-sdk	✅	✅	✅	✅
helix-ota-android	✅	✅	✅	✅
helix-ota-dashboard	✅	✅	✅	✅
helix-update-engine	✅	✅	✅	✅
helix-artifact-validator	✅	✅	✅	✅
helix-rollout-engine	✅	✅	✅	✅
helix-device-identity	✅	✅	✅	✅

CLAUDE.md Required Sections

Every `CLAUDE.md` must contain:

1. **Project Overview** — One-paragraph description of the submodule's purpose
2. **Module Structure** — Package map with responsibility assignments
3. **Coding Conventions** — Language-specific rules (Go: DI via constructors, error wrapping with `%w`, no global state; Kotlin: coroutines for async, Hilt for DI; TypeScript: strict mode, no `any`)
4. **Test Patterns** — Anti-bluff requirements (unique TOKEN, State DELTA, POSITIVE evidence), mock generation commands, test file naming conventions
5. **Common Gotchas** — Platform-specific pitfalls (gomobile: no `chan` in exported types; Android: Binder IPC must run on main thread; Go: `io.Copy` reads until EOF for valid hash)
6. **Dependency Rules** — Which packages may import which other packages (e.g., `handler → service → repository`; never `repository → handler`)
7. **Build Commands** — `make test`, `make lint`, `make build`, `make aar` (if applicable)

AGENTS.md Required Sections

Every `AGENTS.md` must contain:

1. **Agent Assignments** — Which AI agent is responsible for which package
2. **Merge Conflict Zones** — Areas where concurrent agent work is likely to conflict
3. **Interface Stability Guarantees** — Which interfaces are frozen (no breaking changes) vs. evolving
4. **Dependency Update Protocol** — How to bump a vasic-digital or Helix OTA submodule dependency
5. **Release Coordination** — Which submodules must be released before others

---

End of NEW_SUBMODULE_SPECIFICATIONS.md